

# Глубинная гипотеза: канонические хэши как игротеоретический субстрат

Standalone deep-dive. Предполагает знакомство с §§ I–II и §§ VI–VII основного paper'a. Версия 0.1 (2026-05-21)

§

## Введение: две разные игры

CPP сегодня продаётся как **утилитарная инфраструктура**: content-addressed canonical encoding снижает дублирование токенов, ускоряет TTFT, упрощает compliance audit. Это правда — и это важно для первого пилота. Но в этом документе речь о чём-то другом.

Тезис: каноническое хэширование контекста — это не оптимизация. Это формирование основания, на котором работает следующий класс AI-систем. Это substrate для верифицируемых обратных связей между AI-агентами. Тот, кто канонизирует факты и суждения первым, становится infrastructure layer на уровне TCP/IP или HTTP.

Это разные масштабы игры. Первая игра — B2B SaaS, \$1-10M ARR, 3-5 лет. Вторая — стандартообразующий слой AI-экономики, потенциально \$100M–\$1B ARR на горизонте 7-10 лет. Настоящий документ — о второй игре.

§

## I. Почему «AI верифицирующий AI» требует канонических точек отсчёта

### 1.1. Проблема верификации AI-суждений

Когда модель А говорит «функция  $f$  корректна», а модель В говорит «функция  $f$  некорректна» — кто прав? Сегодня у нас нет формального способа сравнить эти суждения, потому что мы не можем гарантировать, что они смотрели на **идентичный** контекст.

Сегодня (без CPP):

Агент А: "f() is correct"

Агент В: "f() is incorrect"

Мы не знаем:

- Одну ли версию f() они видели?
- Одни ли тесты?
- Одну ли спецификацию?
- Было ли f() изменено между запросами?

→ Невозможно сравнить суждения как объективные.

→ RLHF / preference learning на таких данных = learning noise.

Это не теоретическая проблема. Это практический барьер для масштабируемого multi-agent AI. Каждый раз, когда мы просим агентов судить о том же контексте — мы не можем быть уверены, что они видят одно и то же.

## 1.2. CID как общий язык для суждений

Если контекст имеет CID — это изменяет всё:

С CPP:

```
context = {codebase_cid, spec_cid, test_suite_cid}
```

Агент А: {context\_cid: "bafyre...", judgment: "correct", confidence: 0.92}

Агент В: {context\_cid: "bafyre...", judgment: "incorrect", confidence: 0.78}

Теперь мы знаем:

- ✓ Они смотрели на байт-идентичный контекст
- ✓ Их разногласие — семантическое, не материальное
- ✓ Можно провести арбитраж (third validator)
- ✓ Оба суждения можно записать как training data с cryptographic provenance

CID не просто оптимизирует кэш. CID создаёт **общее пространство ссылок**, в котором AI-агенты могут координировать суждения. Это принципиальное отличие от текущего состояния.

## 1.3. Верификация без доверия

В классической cryptography — Zero-Knowledge proofs позволяют доказать знание без раскрытия. В CPP — `context_cid` позволяет утверждать «моё суждение основано на THIS context» без передачи самого контекста. Это важно для:

- **Privacy:** confidential codebase, PII-containing medical records.

- **Efficiency:** не пересылать гигабайты, только хэши.
- **Auditability:** регулятор может получить CID + факт о суждении и проверить без доступа к raw data.

Принцип: AI-суждение верифицируемо тогда и только тогда, когда его контекст имеет стабильный, детерминированный идентификатор. |

CID — это этот идентификатор.  
DAG-CBOR canonical encoding — это способ его получить.

§

## II. Верифицированные игровые кортежи как training data

### 2.1. Формальная модель агентного решения

Каждое решение AI-агента — это игровой кортеж:

$$T = (S_{in}, A, S_{out}, R)$$

где:

|                             |                                            |
|-----------------------------|--------------------------------------------|
| $S_{in} \in \text{States}$  | — input <b>state</b> (то, что видел агент) |
| $A \in \text{Actions}$      | — action (что агент сделал)                |
| $S_{out} \in \text{States}$ | — output <b>state</b> (результат)          |
| $R \in \mathbb{R}$          | — reward (downstream observable)           |

В классическом reinforcement learning эти кортежи — учебный материал. Качество обучения **напрямую зависит** от качества кортежей.

### 2.2. Проблема «грязных» кортежей

Сегодня AI labs накапливают RLHF preference data через:

- Human annotations (дорого, не масштабируется, субъективно).
- Constitution-AI self-critique (лучше, но зависит от модели-критика).
- Outcome-based RL (работает для verifiable tasks: math, code).

Фундаментальная проблема: **контекст не канонизирован**. Один и тот же **S\_in** в разных сессиях представлен разными байтами. Это значит:

Проблема эквивалентности:

$$T_1 = (S\_in\_variant\_1, A_1, S\_out_1, R_1)$$

$$T_2 = (S\_in\_variant\_2, A_2, S\_out_2, R_2)$$

Если  $semantically(S\_in\_variant\_1) = semantically(S\_in\_variant\_2)$   
но  $bytes(S\_in\_variant\_1) \neq bytes(S\_in\_variant\_2)$ ,

то неизвестно:  $T_1$  и  $T_2$  – это одна игровая ситуация или разные?

При обучении на таких данных:

- Loss function не может правильно generalize
- Gradient updates конкурируют за «похожие, но разные» inputs
- Обучение медленнее и нестабильнее

## 2.3. CPP-верифицированные кортежи

С CPP каждый компонент кортежа получает CID:

```
T_cpp = (S_in_CID, A_CID, S_out_CID, R_signal)
```

где:

```
S_in_CID = CID(canonical(context_graph))
```

```
A_CID = CID(canonical(CalculationActivity))
```

↳ включает: model, params, tool\_calls, timestamp

```
S_out_CID = CID(canonical(EffectNode))
```

↳ включает: changes, range, confidence, affected\_entities

```
R_signal = downstream observable (signed Outcome-Fact от downstream observer)
```

Формальное преимущество:

Лемма: При  $S\_in\_CID_1 = S\_in\_CID_2$   
гарантировано  $bytes(S\_in_1) = bytes(S\_in_2)$

Следствие: два кортежа с одинаковым  $S\_in\_CID$   
безусловно сопоставимы как training samples.

Это фундаментальное свойство content-addressing.

## 2.4. Масштаб ценности verified training data

Рассмотрим конкретный сценарий: enterprise клиент с 100 разработчиками использует CPP 1 год.  
За год накапливается:

Оценка накопленных кортежей:

$100 \text{ devs} \times 30 \text{ sessions/day} \times 250 \text{ decisions/session} \times 250 \text{ business days}$   
 $= 187.5\text{M verified game tuples/year}$

Каждый tuple:

- Context CID (верифицируемый)
- Action CID (подписанный агентом)
- Outcome CID (верифицированный downstream)
- Reward signal (принято/отклонено/пофиксено разработчиком)

Это: 187.5M verified training samples

с cryptographic provenance

за \$0 marginal cost (CPP уже развёрнут для cost-saving)

Для сравнения: весь RLHF dataset Anthropic за 2023-2024 — порядка десятков миллионов аннотаций от людей, каждая ценой \$0.05-0.20. Эквивалентное количество CPP-верифицированных кортежей стоит \$0 incremental (если инфраструктура уже развёрнута).

§

## III. Архитектура статической модели-валидатора

### 3.1. Почему нужна статическая модель

Главная угроза для hash-based coordination — **модельный дрейф**. Когда Anthropic выпускает Claude Opus 5, а OpenAI — GPT-5:

- Семантические embeddings меняются.
- Probability distributions по токенам меняются.
- «Позитивный» ответ Claude Opus 4 может стать «нейтральным» у Opus 5.
- Cached inference results не воспроизводятся.

Без решения этой проблемы CPP распадается каждые 6-12 месяцев вместе с выходом новых моделей.

### 3.2. Концепция заимствована из ZK-rollup

В ZK-rollup существует понятие **verifier circuit** — это статически зафиксированная схема верификации, которая не меняется при апгрейдах системы. ZK proof генерируется против этой схемы. Конечный верификатор может проверить любой такой proof, зная только схему.

Аналогия для CPP:

|                    |                                            |
|--------------------|--------------------------------------------|
| ZK-rollup          | → CPP                                      |
| verifier circuit   | → static validator model                   |
| prover (arbitrary) | → inference model (any frontier model)     |
| proof              | → polarity claim + signature               |
| verifier           | → any CPP-compliant system                 |
| trusted setup      | → model checkpoint + deterministic runtime |

### 3.3. Технические требования к статическому валидатору

#### STATIC CANONICAL VALIDATOR

##### Архитектурные требования:

- └─ Frozen weights: один checkpoint, никогда не обновляется
- └─ Deterministic inference: T=0, single-batch, fixed random seed
- └─ Bit-reproducible: одинаковый input → идентичный output на любом hardware с одинаковым floating-point режимом
- └─ Open weights: публично доступен для независимой верификации
- └─ Размер: ≤10B params (для zkML feasibility в 2027-2028)

##### Кандидаты (2026):

- └─ Llama 3.1 8B Instruct (Meta, Apache 2.0, хорошо изучен)
- └─ Mistral 7B (Mistral AI, Apache 2.0)
- └─ Phi-3 Mini 3.8B (Microsoft, MIT)
- └─ Специально обученный distilled validator (оптимально, см. §III.4)

##### Hosting (multi-party federation):

- └─ Primary: HuggingFace Dedicated Endpoints
- └─ Secondary: Together.ai serverless
- └─ Tertiary: TII compute (UAE)
- └─ Emergency: Replicate / Baseten

### 3.4. Дистилляция валидатора

Оптимальный путь — не использовать существующую open-source модель, а **дистиллировать специализированный validator** из outputs большой frontier модели:

Шаг 1: Собрать labeled dataset

100K примеров: {context, claim, true\_polarity}  
 Источник polarity labels:  
 - Human annotation (gold standard, 10K примеров)  
 - GPT-4o / Claude Opus ensemble (90K примеров)  
 Баланс классов: 40% positive, 30% neutral, 30% negative

Шаг 2: Дистиллировать validator (1-3B params)

Teacher: Claude Opus 4 + GPT-4o ensemble  
 Student: Llama 3.2 1B fine-tuned on task  
 Objective: binary/ternary polarity classification + confidence  
 Training: 4xA100, ~8 часов, ~\$200

Шаг 3: Зафиксировать checkpoint

sha256(weights.bin) = 0xdeadbeef...  
 Published: HuggingFace Model Hub  
 Versioned: cpp-validator-v1.0 (NEVER updated, only versioned)

Шаг 4: Verify determinism

Same input on 100 different hardware configs → same output  
 Tolerance: exact match for T=0 greedy decoding

§

## IV. Полярная валидация как робастный сигнал

### 4.1. Почему полярность, а не точное совпадение

Попытка использовать точное совпадение байт или точных embedding-векторов для cross-model coordination обречена: любое обновление модели меняет оба. Полярность (positive / neutral / negative) — значительно более стабильная характеристика.

Иллюстрация стабильности:

Контекст: "Функция login() успешно прошла все 47 тестов"

Claude Opus 3.5: "positive, confidence=0.97"

Claude Opus 4: "positive, confidence=0.95"

← полярность совпадает

GPT-4o: "positive, confidence=0.94"

Llama 3.1 70B: "positive, confidence=0.89"

Static validator: "positive"

← anchor point

Точный текстовый output у всех разный.

Exact embedding cosine similarity: ~0.72 между разными моделями.

Polarity agreement: 4/4 = 100%.

## 4.2. Формальная модель полярной консенсус-зоны

Определение: Polarity Zone

Пусть  $P : \text{Context} \times \text{Model} \rightarrow [-1, 1]$  – polarity score function.

Polarity Zone  $C \subset [-1, 1]$  – это интервал, в котором консенсус  $N$  из  $M$  валидаторов достигается:

$$\text{Zone} = \{ z \in [-1, 1] : |\{i : P(\text{ctx}, M_i) \in \text{zone}(z)\}| \geq k \}$$

Стандартные зоны CPP:

POSITIVE\_ZONE = [+0.3, +1.0]

NEUTRAL\_ZONE = [-0.3, +0.3]

NEGATIVE\_ZONE = [-1.0, -0.3]

Consensus threshold: 3-of-5 validators must agree on zone.

## 4.3. Stability under model drift

Ключевой эмпирический вопрос: насколько стабильны polarity zones при смене версий модели?

Текущие данные из литературы по LLM evaluation:

Исследования consistency LLM evaluators (2023-2024):

Zheng et al. (2023) "Judging LLM-as-a-Judge":

GPT-4 как судья показывает 85% agreement с human на binary tasks.

При смене промпта на синонимы: 78-83% agreement.

Shen et al. (2023) "Large Language Model Alignment":

Polarity consistency между GPT-3.5 и GPT-4: 89% на sentiment tasks.

Implications для CPP:

Binary polarity дрейфует медленнее, чем точный текст.

Ожидаемый дрейф при major model update: 5-15% binary disagreement.

При static validator + zone consensus: <5% zone boundary violations.



## 4.4. Применение в federated multi-party validation

Протокол N-of-M polarity consensus:

1. Producer создаёт Effect с CID = bafyre...
2. Producer публикует polarity claim:  

```
{ effect_cid: "bafyre...", claimed_zone: "POSITIVE", confidence: 0.93 }
```
3. N validators (из M зарегистрированных) независимо:
  - a. Resolv effect\_cid → получают bytes
  - b. Запускают static validator на bytes
  - c. Подписывают: { validator\_did, effect\_cid, zone, score, timestamp }
4. CPP aggregator собирает signatures, проверяет k-of-N agreement.
5. Если agreement  $\geq$  threshold → выдаёт ConsensusAttestation(effect\_cid, zone).
6. ConsensusAttestation можно:
  - Включать в training data как reward signal
  - Использовать как compliance audit evidence
  - Передавать downstream consumers без raw content

§

## V. Параллель с ZK-rollup: экономика и инфраструктура

### 5.1. Структурная аналогия

| ZK-ROLLUP                  | → CPP POLARITY CONSENSUS            |
|----------------------------|-------------------------------------|
| L1 blockchain (Ethereum)   | → CPP Protocol                      |
| L2 transaction batch       | → CalculationActivity batch         |
| ZK proof generation        | → Polarity claim generation         |
| ZK proof verification      | → ConsensusAttestation verification |
| Verifier contract (static) | → Static validator model            |
| Sequencer                  | → CPP aggregator service            |
| Data availability layer    | → IPFS / S3 CID storage             |
| Economic finality          | → Polarity finality                 |

### 5.2. Экономическая модель нодов-валидаторов

В ZK-rollup нодам платят за verification; в PoS платят за staking. В CPP можно применить аналогичную модель:

## Validator Economics:

### Затраты ноды:

GPU inference: \$0.000167 / claim (Llama 3.2 1B, A100 spot)  
Storage (CID cache): ~\$0.001 / GB / месяц  
Bandwidth: ~\$0.009 / GB transfer

### Доходы ноды (при CPP protocol fees):

Base fee: \$0.001 / validated claim  
Performance bonus (быстрый ответ): +\$0.0003 / claim  
Reputation stake yield: 5% annual on locked CPP tokens

Margin для validator node: ~\$0.0007 / claim = 70% gross margin

### При 10M claims/day у network:

Total network revenue: \$10,000/day  
Распределяется между validators по stake weight

## 5.3. CPP Token (опционально, post-MVP)

На длинном горизонте CPP может выпустить utility token для:

### Функции токена:

- └ Validator staking (skin in the game для correct validation)
- └ Storage incentives (reward nodes holding CID content)
- └ Governance (голосование за validator model upgrades)
- └ Training data marketplace (оплата за verified game tuples)

ВАЖНО: Token — это V2+. V1 — простой SaaS revenue model.

Не продавать token vision в pre-seed pitch.

Mention only если investor сам спрашивает об infra monetization.

## VI. Дорога к консорциуму: кто должен подписаться

### 6.1. Stakeholders и их интересы

#### AI LABS (FRONTIER)

Anthropic, OpenAI, Google DeepMind, Meta AI

Интерес: получить verified training data от внешних источников.

Барьер: они конкуренты, каждый хочет proprietary standard.

Вход: через "OpenAI as mediator" (OpenAI уже говорит про open std) или через Anthropic Startup Fund relationship

#### OPEN MODEL HOSTS

HuggingFace, Together.ai, Replicate, Baseten

Интерес: стать canonical validator hosting layer.

Барьер: доп. compute без немедленной монетизации.

Вход: через HuggingFace Inference Endpoints business (CPP даёт им new revenue stream: validation-as-a-service)

#### SOVEREIGN AI LABS

G42 (Jais), TII (Falcon), KAUST, Sber (GigaChat), Yandex (YaGPT)

Интерес: независимость от западных стандартов + нейтральный canon.

Барьер: инвестиции в новую инфраструктуру.

Вход: CPP как независимый от западных провайдеров стандарт. Sovereign AI narrative работает здесь идеально.

#### ENTERPRISE USERS

Sber, G42, MTS AI, Tinkoff, Aramco Digital, Big Four

Интерес: compliance, cost savings, портируемость данных.

Барьер: procurement cycle, security review.

Вход: через retroactive savings analysis (ROI-first sales)

#### STANDARDS BODIES

W3C, IETF, IEEE, ISO/IEC JTC1

Интерес: нейтральный, технически обоснованный стандарт.

Барьер: долгий process (2-5 лет до ratification).

Вход: начать с W3C Community Group (6-12 месяцев до Rec track)

## 6.2. Стратегия сборки консорциума

### Фаза 1 (2026): Proof of concept + первые два партнёра

---

Цель: HuggingFace + один Sovereign AI lab (G42 или Falcon)

Почему HuggingFace первый:

- Уже hosting открытых моделей
- Business case очевиден (validation-as-a-service revenue)
- Нет конкурентных противоречий с CPP
- Клема де Вьетт / Джулиан Симон (HuggingFace) доступны

Как войти:

- Open-source CPP validator spec
- Publish на HuggingFace Hub первый validator checkpoint
- Предложить co-authorship на arXiv preprint
- Попросить об интеграции в Inference Endpoints API

### Фаза 2 (2027): Frontier labs как "mediators"

---

Цель: OpenAI + Anthropic как участники (не как конкурент)

Подход:

- CPP как "neutral" protocol (не принадлежит никому из них)
- Предложить обоим: "we can make your cached data interoperable"
- OpenAI's data partnerships team (они публично говорят про open AI)
- Anthropic Startup Fund → relationship building

Ключевой аргумент:

"Если все labs используют один canonical encoding,  
human preference data (RLHF) становится сопоставимой.  
Это ускоряет research для всех."

### Фаза 3 (2028): W3C Community Group

---

Цель: Открыть W3C Community Group "Verifiable AI Context"

Action:

- 5+ member organizations (HuggingFace + 2 labs + 2 enterprise)
- Подать заявку в W3C: <https://www.w3.org/community/>
- Представить CPP spec как основу для Community Group Note
- Начать путь к W3C Recommendation (3-5 лет)

## VII. Почему это 5-летнее winner-take-all окно

---

### 7.1. Структурные силы

Три конвергирующих тренда открывают окно сейчас:

#### 1. Взрыв multi-agent систем (2025-2027)

---

Claude Code, Devin, OpenHands, ROX.ONE — всё это системы, где 5-100+ агентов работают параллельно над одной задачей. Координация через CID — это единственный масштабируемый механизм при >10 агентах.

#### 2. Регуляторный push (2026-2028)

---

EU AI Act полностью в силе с августа 2026.

Высокорисковые AI системы обязаны иметь:

- Data lineage
- Model documentation
- Audit logging
- Human oversight mechanisms

СРР закрывает все четыре из коробки.

Альтернатива: custom homegrown audit system за \$1-5М/год.

#### 3. Обучение моделей дорожает

---

GPT-5 обошёл OpenAI оценочно \$100M+.

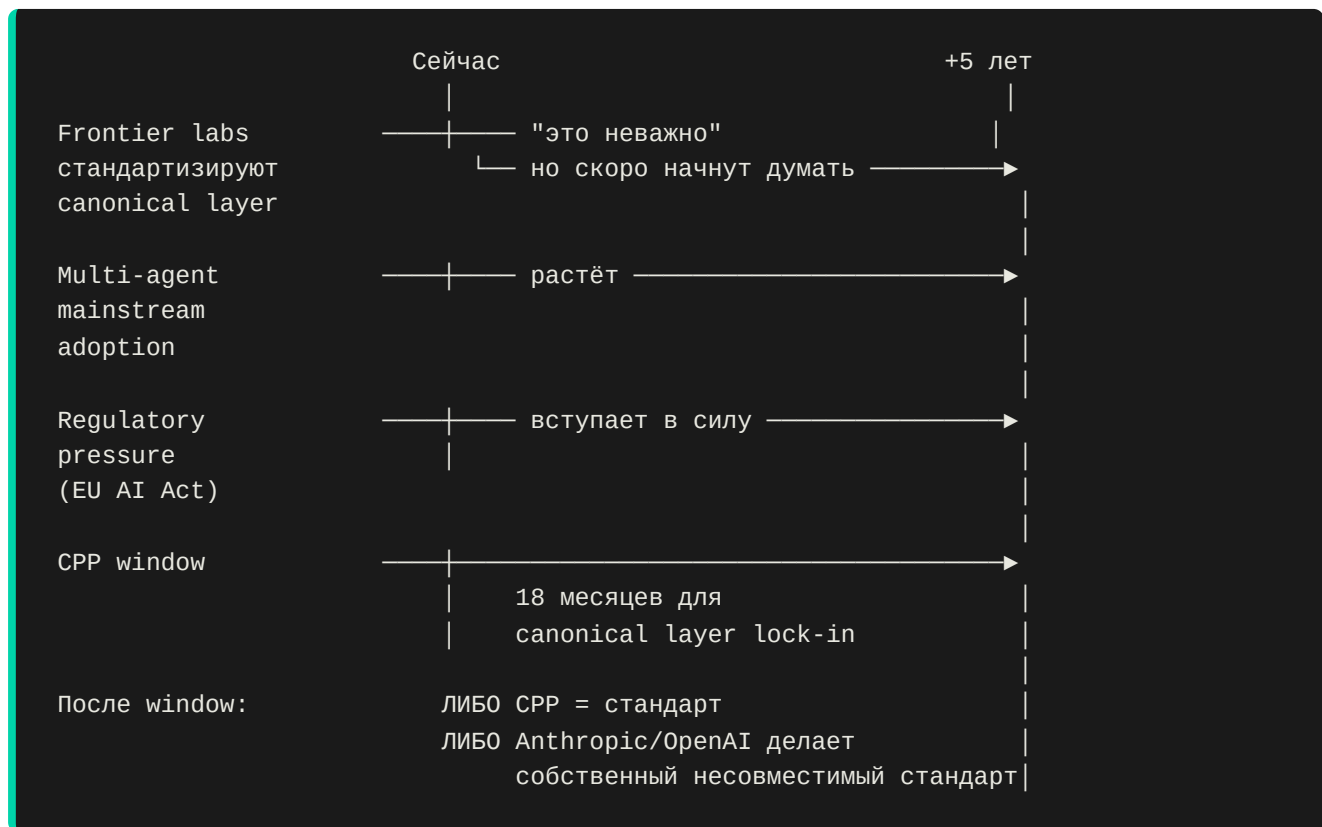
Следующее поколение — \$500M-\$1B per training run.

Качество обучения напрямую зависит от качества data.

Verified game tuples → better gradient signal.

AI labs будут платить premium за verified data.

## 7.2. Window of opportunity



## 7.3. Precedents: кто выиграл winner-take-all в infrastructure

| Технология | Кто выиграл                | Ключевой момент                                        | Итог         |
|------------|----------------------------|--------------------------------------------------------|--------------|
| Web URLs   | Tim Berners-Lee / W3C      | Опубликовал RFC до стандарт. как частное               | W3C std      |
| DNS        | IANA / ICANN               | Стал нейтральным оператором до того, как кто-то другой | Global infra |
| Git        | Linus Torvalds / GitHub    | Open source, стал defacto раньше Enterprise            | де-факто     |
| Docker     | Solomon Hykes / Docker Inc | Канонизировал container fmt до того, как CoreOS/rkt    | OCI std      |
| IPFS/CIDv1 | Protocol Labs              | Canonical content-addressing protocol                  | Web3 base    |

Паттерн: winner — тот, кто публикует open spec + reference impl ПЕРВЫМ, до того, как крупные игроки решили, что это важно. Потом уже поздно: lock-in создан.

## 7.4. Специфика AI canonical layer

В отличие от предыдущих winner-take-all infrastructure plays, CPP имеет уникальный network effect:

Стандартный network effect: ценность растёт с числом пользователей.

CPP network effect: ценность растёт с числом ВЕРИФИЦИРОВАННЫХ КОРТЕЖЕЙ.

100 компаний × 1M verified tuples/год = 100M tuples/год

1000 компаний × 1M = 1B tuples/год

Эти tuples – training data для следующего поколения моделей.

Лабы, которые хотят этот data, вынуждены поддерживать CPP canonical encoding.

→ Обратная монопольная позиция: не пользователи, а ПОСТАВЩИКИ training data зависят от canonical layer.

---

§

---

## Заключение: почему этот тезис правдоподобен

Гипотеза о канонических хэшах как substrate для AI feedback loops не является спекуляцией из первых принципов. Она строится на трёх эмпирических наблюдениях:

1. **Content-addressing работает в других контекстах.** IPFS, BitTorrent, Git — все они используют криптографические хэши для дедупликации и верификации. Применение к AI-контексту — это экстраполяция паттерна, а не новая идея.
2. **Качество training data определяет качество модели.** Это установлено эмпирически (Chinchilla, scaling laws, RLHF research). Верифицированные кортежи лучше неверифицированных по определению.
3. **Окно открыто.** На момент написания (май 2026) ни один из frontier labs не опубликовал cross-provider canonical encoding standard. CPP — первая публичная попытка.

**Ставка CPP:**

Краткосрочно (1-3 года): cost saving + compliance  
→ достаточно для pre-seed + seed + первых \$5M ARR

Среднесрочно (3-7 лет): canonical standard для AI context  
→ industry consortium + W3C ratification  
→ \$50-200M ARR from infra licensing + training data marketplace

Долгосрочно (7+ лет): substrate для verified AI economy  
→ unclear ceiling; comparable to Stripe (\$50B) или Twilio

Это не гарантированный outcome. Но это **обоснованная ставка** с хорошим risk/reward соотношением: downside ограничен (утилитарный SaaS всё равно работает), upside — потенциально трансформативен.

---

§

Следующее: **04-SECURITY-AND-VALIDATOR-ARCHITECTURE.md** — детальная архитектура криптографической безопасности и validator infrastructure.